Technical Write-up

Architecture

The design isolates model reasoning from final policy authority. All three agents have distinct contracts, tools, and persisted state, but an agent does not need to spend tokens on every run. The Extractor always uses vision in real mode. The Validator calls the model only for genuinely ambiguous textual mismatches; exact matches, missing evidence, and low confidence are resolved safely by rules. The Router calls the model only for exception reasoning or an amendment draft; clean auto-approval is deterministic. Python guards independently enforce schema, confidence, customer rules, and allowed outcomes. Each stage consumes and emits typed JSON. LangGraph orchestrates the three nodes and its SQLite checkpointer persists compact graph state under a stable thread ID; document bytes live in a hash-addressed local upload store rather than being copied into every checkpoint. The application database separately stores the content hash, provider, stage, extraction, validation, route, error, and timestamps for the operator audit trail. Gemini processes PDFs natively with JSON Schema output; Groq remains an image-vision fallback.

Three nastiest failure modes

1. Plausible hallucination on an unreadable scan

The dangerous failure is a valid-looking HS code invented from context. The extractor is instructed not to infer; every value requires a source snippet and page. The schema checker reduces any value without evidence below the approval threshold. The validator marks missing or low-confidence fields `uncertain`, and the router cannot auto-approve them. In the messy sample, the obscured HS code and illegible Incoterm surface for review.

2. Correct extraction, wrong customer rule

A perfect model can still produce a bad outcome if policy is stale. Rules are external JSON, customer-scoped, and treated as versioned configuration rather than prompt prose. The UI shows both found and expected values. Production records must also store `rule_set_id` and version so a decision can be reconstructed. Rule changes require four-eyes approval and replay against an offline regression set.

3. Partial pipeline failure creates a duplicate or lost decision

Model calls can time out after work has succeeded. The input hash and run ID make processing idempotent; stage checkpoints separate extracted, validated, and routed states. Retries are bounded and never repeat deterministic stages unnecessarily. A failed state is visible and cannot be mistaken for approval. In production, a queue with a dead-letter state would replace the in-process runner.

Observability at 50 customers

Every run needs a correlation ID from ingestion through model call, validation, route, storage, and UI. Structured logs should include customer, shipment, document hash, document type, stage, model/version, rule-set version, attempt, latency, token usage, estimated cost, confidence summary, outcome, and error class. Sensitive document content and full values should not enter normal logs.

The operating dashboard would show volume and success rate by customer; p50/p95 stage latency; cost per document; schema failures; uncertain/mismatch rates by field, document type, supplier, and layout; retry/dead-letter counts; confidence drift; human correction rate; and any false approval as a page-level incident.

A support engineer can search one correlation ID and reconstruct the complete state transition and evidence used.

Cost

Back-of-envelope for a 1–3 page document: input rendering/vision tokens dominate, while output is small structured JSON. At an illustrative blended multimodal rate, a normal document should remain in the low cents, but high-resolution multi-page scans can increase cost several-fold. The exact figure is measured from API usage, not hard-coded, because model pricing changes.

Controls: reject duplicate hashes, crop/render only relevant pages when safe, choose the smallest model that clears the eval gate, cap pages and file size, cache extraction by content/model/schema version, and escalate only ambiguous cases. A clean run makes one model call (extraction); validation and routing are local. Never retry a deterministic policy failure with a larger model.

Latency

The vision extraction call is the slowest hop; validation, routing, and SQLite are typically milliseconds. Improve perceived latency with visible stage updates. Improve actual latency by rendering pages once, parallelizing independent document attachments later, using a smaller first-pass model, and escalating only hard cases. Keep hard timeouts and a queue so slow calls do not block other work.

With a week instead of a day

I would add a durable queue and resumable workflow engine; JSON Schema contracts; rule-set versioning; document malware scanning and retention controls; encrypted object storage; authentication and customer isolation; a 100-document labelled eval harness; model-usage telemetry; cross-document consistency checks; and Playwright/API tests. I would not add autonomous emailing. The next product risk is safe integration into CG review, not generative autonomy.